THE FRAMEWORK YOU GRADUATE TO, NOT START WITH

The LangGraph Cheat Sheet

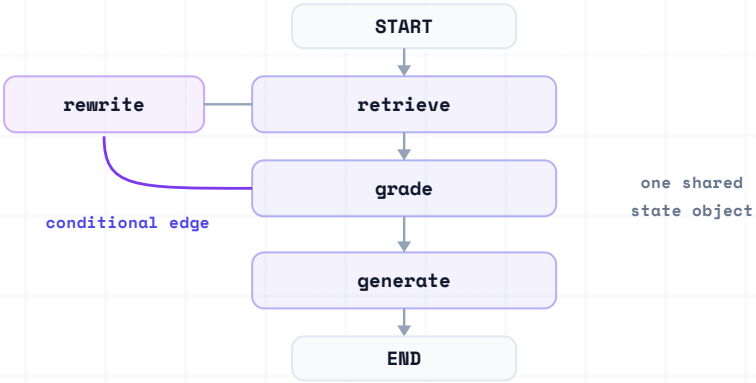
Nodes, edges, state and checkpoints. Everything you need to build an agent that loops, retries and survives a restart.

PRACTICAI

Nodes, Edges, State

MENTAL MODEL

LangGraph is a state machine for LLM work. Three concepts, and everything else is built from them.



CONCEPT	IS
Node	A plain function that takes state and returns a partial update
Edge	Where control goes next, fixed or conditional
State	One typed dict every node reads from and writes to

> A chain runs start to finish. A graph can go backwards, and that single difference is the whole reason this library exists.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

PRACTICAI

Define the State First

BACKBONE

Every node signature depends on this, so it is the first thing you write and the thing you change most carefully.

EVERY NODE READS AND WRITES ONE DICT

question	str
docs	List[Document]
answer	str
retries	int

a node returns a partial dict, LangGraph merges it in

```python
from typing import TypedDict, Annotated, List
import operator

class State(TypedDict):
    question: str
    docs: List[str]
    answer: str
    retries: int
    log: Annotated[list, operator.add]    # appends instead of replacing
```

+ A reducer appends instead of overwriting

Keep the retries counter in state

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

PRACTICAI

A Graph in 15 Lines

MINIMAL

The smallest complete graph. Everything else in this deck is added to this shape.

```python
from langgraph.graph import StateGraph, START, END

def retrieve(state: State) -> dict:
    return {'docs': retriever.invoke(state['question'])}

def generate(state: State) -> dict:
    return {'answer': llm.invoke(prompt(state)).content}

g = StateGraph(State)
g.add_node('retrieve', retrieve)
g.add_node('generate', generate)
g.add_edge(START, 'retrieve')
g.add_edge('retrieve', 'generate')
g.add_edge('generate', END)
app = g.compile()
app.invoke({'question': 'what is the refund window?'})
```

- START and END are built in sentinels
- compile() validates the graph before you run it

> Get this running first. Every feature that follows is one line added to this file.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Writing a Node

FUNCTIONS

A node is an ordinary Python function. Return only the keys you changed, and LangGraph merges them into state for you.

```
def grade(state: State) -> dict:
    verdict = grader.invoke(state['docs'], state['question'])
    return {'grade': verdict}      # partial update, not the whole
state
```

RULE	WHY
Return a partial dict	Merging is handled for you, do not rebuild state
Keep nodes pure	A node you can call directly is a node you can test
One responsibility each	Small nodes give you useful retry boundaries
Raise on real errors	Let the graph retry policy handle it

A pure node is a testable node

Small nodes give useful retry boundaries

> If a node needs three things from state and writes five, it is really three nodes and you will regret merging them.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Branching

CONDITIONAL EDGES

This is the whole reason to use LangGraph. A router function inspects state and returns the name of the next node.

```python
def route(state: State) -> str:
    if state['grade'] == 'useful':
        return 'generate'
    if state.get('retries', 0) >= 2:
        return 'give_up'           # always have an exit
    return 'rewrite'

g.add_conditional_edges('grade', route, {
    'generate': 'generate',
    'rewrite':  'rewrite',
    'give_up':  END})
```

PIECE	ROLE
The router function	Pure logic, no side effects, easy to test
The mapping dict	Router output to node name
An END branch	The escape hatch, always include one

> Every loop needs a counter in state and an escape route. A graph without one will run until your budget does not.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Cycles

SELF CORRECTION

An edge pointing backwards is a loop. This is what a plain chain cannot do, and what makes self correcting RAG possible.

```
def rewrite(state: State) -> dict:
    better = llm.invoke(f'Rewrite for search: {state["question"]}')
    return {'query': better.content,
            'retries': state.get('retries', 0) + 1}

g.add_edge('rewrite', 'retrieve')    # back around the loop
```

GUARD	STOPS
A retries counter in state	An infinite rewrite loop
recursion_limit on compile	A runaway graph, as a backstop
A give_up branch	Returning nothing instead of a clear failure

A backwards edge is all a cycle is

Count retries in state, never in a global

> Set recursion_limit even when you have a counter. It is the seatbelt for the bug you have not written yet.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Checkpointers MEMORY

A checkpointer saves state after every node. That single line gives you conversation memory, resume after crash, and time travel debugging.

```python
from langgraph.checkpoint.memory import MemorySaver

app = g.compile(checkpointer=MemorySaver())
cfg = {'configurable': {'thread_id': 'user-42'}}

app.invoke({'question': 'what is the refund window?'}, cfg)
app.invoke({'question': 'and for digital goods?'}, cfg)  # remembers
```

SAVER	USE FOR
MemorySaver	Development and tests, gone on restart
SqliteSaver	Single node production, zero setup
PostgresSaver	Real production, shared across workers

Multi turn memory for free

Resume a run after a crash

> thread_id is the conversation. Get it wrong and two users share a memory, which is a data leak, not a bug.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Watching It Run STREAM

Three streaming modes, and they answer different questions. Most teams want values for the UI and updates for debugging.

```
for chunk in app.stream(inp, cfg, stream_mode='updates'):
    print(chunk)          # what each node just changed

for chunk in app.stream(inp, cfg, stream_mode='values'):
    print(chunk['answer'])  # full state after every node
```

MODE	EMITS
updates	Only the delta each node returned
values	The entire state after each node
messages	Token by token from the LLM inside a node
debug	Everything, for when you are stuck

values for the user interface

updates for your logs

> Streaming updates into your logs during development will teach you more about your graph than any amount of reading the code.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Human in the Loop INTERRUPTS

Pause the graph before a dangerous node, let a person look, then resume. This is the feature that makes agents safe to deploy.

```
app = g.compile(checkpointer=saver,
                interrupt_before=['send_email'])

app.invoke(inp, cfg)              # stops before send_email
state = app.get_state(cfg)      # a human reviews it
app.update_state(cfg, {'draft': edited})
app.invoke(None, cfg)             # resume from where it stopped
```

- Interrupt before anything irreversible
- update_state lets a human correct the draft
- invoke(None, cfg) continues the run
- Requires a checkpointer, it has nowhere to pause

> This is the feature that lets an agent touch production systems. Without it, every irreversible action is an act of faith.

Interrupt before sending, paying, deleting or writing.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Prebuilt Agents SHORTCUTS

You do not have to hand build the tool calling loop. The prebuilt agent covers the common case in one line.

```python
from langgraph.prebuilt import create_react_agent, ToolNode

agent = create_react_agent(llm, tools=[search, calculator])
agent.invoke({'messages': [('user', 'what is our refund window?')]})

# or use ToolNode inside your own graph
g.add_node('tools', ToolNode([search, calculator]))
```

PREBUILT	GIVES YOU
`create_react_agent`	The full think, call tool, observe loop
`ToolNode`	Just the tool execution step, inside your graph
`tools_condition`	The standard router for tool calls

One line to a working tool agent

You give up per step control to get it

> Start with create_react_agent. Hand build a graph the moment you need a step it does not give you, usually validation.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Subgraphs COMPOSITION

A compiled graph is itself a node. That is how you keep a large system readable instead of ending up with forty nodes on one canvas.

```
research = research_builder.compile()
writing  = writing_builder.compile()

parent = StateGraph(State)
parent.add_node('research', research)   # a graph as a node
parent.add_node('writing',  writing)
```

- Each subgraph is testable on its own
- Different teams can own different subgraphs
- Shared state keys pass through automatically
- The parent graph stays readable

> If your graph does not fit on one screen, it wants to be two graphs.

Subgraphs are how a multi team agent system stays reviewable.

> Compile a subgraph once and reuse it in several parents. It is the only composition primitive you need.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Fan Out and In

CONCURRENCY

Two edges from one node run in parallel. The catch is what happens when both write the same state key.

```
g.add_edge('plan', 'search_web')
g.add_edge('plan', 'search_docs')   # these run together
g.add_edge('search_web',  'merge')
g.add_edge('search_docs', 'merge')

# state key must know how to combine concurrent writes
results: Annotated[list, operator.add]
```

- Parallel branches cut wall clock time
- operator.add is the usual reducer for lists
- No reducer on a shared key is an error
- Cost does not drop, only latency

> Without a reducer, two parallel nodes writing one key is an error. That error is LangGraph protecting you from a silent lost update.

Parallel branches cut latency, never cost. Both calls still happen.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Retries and Errors ROBUSTNESS

Provider calls fail. Handle it at the node level so one bad request does not lose the whole run.

```python
from langgraph.pregel import RetryPolicy

g.add_node('generate', generate,
           retry=RetryPolicy(max_attempts=3))

app = g.compile(checkpointer=saver)
app.invoke(inp, {**cfg, 'recursion_limit': 25})
```

FAILURE	WHERE TO HANDLE IT
Transient API error	A node retry policy
Bad tool output	Inside the tool, return a readable string
Wrong branch taken	A grading node before the branch
Runaway loop	recursion_limit plus a counter

> With a checkpointer, a failed run is resumable rather than lost. That turns a total failure into a retry from the node that broke.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Seeing the Graph INSPECTION

You can draw the graph and replay any past run. Both are worth setting up before you need them.

```
print(app.get_graph().draw_ascii())
app.get_graph().draw_mermaid_png()

# time travel: every checkpoint is addressable
for snap in app.get_state_history(cfg):
    print(snap.next, snap.values.keys())
app.invoke(None, past_snapshot.config)   # rerun from there
```

- Draw the graph in your README
- Replay a failed run from any node
- get_state shows exactly where it stopped
- Most bugs are a wrong edge, and the drawing shows it

> Time travel is the feature nobody uses until their first production incident, and then it becomes the only thing they want.

Draw the graph once and put the picture in your README.

Self Correcting RAG RECIPE

The canonical LangGraph application, and the one worth building first. Retrieve, grade, rewrite if weak, then generate.

```
g.add_edge(START, 'retrieve')
g.add_edge('retrieve', 'grade')
g.add_conditional_edges('grade', route,
    {'generate': 'generate', 'rewrite': 'rewrite', 'give_up': END})
g.add_edge('rewrite', 'retrieve')
g.add_edge('generate', END)
```

Grading catches bad retrieval before you pay to generate

Rewriting fixes vague and pronoun heavy questions

The give_up branch returns an honest I do not know

> Build this one first. It is small enough to finish in an afternoon and it exercises every concept in this deck.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

PRACTICAI

What a Graph Costs

BUDGET

Every node that calls a model is a bill. Loops multiply that, so know your worst case before you deploy.

NODE TYPE	COST PER RUN
Retrieval	Effectively free, one embedding call
Grading	One cheap model call, worth every cent
Rewrite	One call, times the retry count
Generation	The expensive one, usually 90 percent of the bill
Checkpointing	Storage only, no model call

Worst case is nodes times max retries

Use a cheap model for grading and routing

> Grading with a small model to avoid one unnecessary generation with a large one pays for itself many times over.

Know your worst case before you deploy, not after the invoice.

> Worst case is nodes times max retries. Multiply it out once, on paper, before the first deploy.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Testing a Graph QUALITY

Nodes are plain functions, so test them like plain functions. Then test the routing separately from the nodes.

```python
def test_route_gives_up():
    assert route({'grade': 'bad', 'retries': 2}) == 'give_up'

def test_retrieve_node():
    out = retrieve({'question': 'refund window'})
    assert len(out['docs']) > 0
```

- Routers are pure functions, test every branch
- Mock the LLM when testing control flow
- Run the whole graph a few times for a pass rate
- Temperature 0 for deterministic assertions

> Testing the router separately from the nodes is what makes a graph testable at all. Routers are pure functions with no excuse.

Mock the model and your control flow tests become deterministic.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

What Goes Wrong PITFALLS

Six failure modes cover almost every LangGraph problem, and none of them are the library misbehaving.

MISTAKE	SYMPTOM
`Returning full state from a node`	Overwrites keys other nodes set
`No loop counter`	A graph that never terminates
`No checkpointer`	Interrupts and memory silently do nothing
`Parallel writes, no reducer`	A runtime error at merge time
`Fat nodes`	Impossible to retry or test in isolation
`Reused thread_id`	One user seeing another user history

THE TWO THAT COST REAL MONEY

∞ No loop counter, an unbounded bill

A shared thread_id, a data leak

> Five of these six are caught by reading the graph drawing once. Draw it before you debug it.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

When to Use It FIT

LangGraph is the right answer for a specific shape of problem, and overhead for everything else.

USE LANGGRAPH

- The flow loops or branches
- You need retries per step
- A human approves mid run
- State must survive a restart

DO NOT BOTHER

- One retrieval, one answer
- A fixed linear pipeline
- Latency is very tight
- A plain function is clearer

→ Linear flow, use a function

Branching flow, use a graph

> A graph with no cycles and no conditional edges is a chain with extra syntax. Use LangGraph when the flow is genuinely a graph.

SIGNAL	VERDICT
A step may need to repeat	Use a graph
A human must approve mid run	Use a graph
State must survive a restart	Use a graph
None of the above	A function is clearer and faster

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

The API You Will Use CHEAT SHEET

Almost everything you write will come from this list.

CALL	DOES
`StateGraph(State)`	Start a builder against a state schema
`add_node(name, fn)`	Register a function as a step
`add_edge(a, b)`	Unconditional transition
`add_conditional_edges(n, fn, map)`	Branch on a router result
`compile(checkpointer=, interrupt_before=)`	Build the runnable app
`invoke / stream / astream`	Run it, optionally streaming
`get_state / update_state`	Inspect or edit a paused run
`get_state_history`	Every checkpoint, for replay

THE THREE YOU WILL FORGET

- recursion_limit
- Annotated reducers
- thread_id per user

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

PRACTICAI

The LangGraph Toolkit

- Nodes
- Edges
- Typed state
- Conditional edges
- Loops and guards
- Checkpointers
- Interrupts
- Subgraphs
- Parallel nodes
- Time travel

Anyone can draw a graph.
Engineers know where
the loop exits.

Save this before your next agent build.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT